

trace-logic-lean: Formal Trace Logic for Finite Markov Chains in Lean 4

Ben Cassie

June 2026

Abstract

The repository **trace-logic-lean** formalises a finite-state trace operation for Markov chains in Lean 4 and Mathlib. The trace operation observes a visible block while marginalising hidden states through a Schur-complement expression. The development proves row-stochasticity of the trace, the trace tower property, transitivity of a trace-semantic order, and transient-hidden-block corollaries that discharge the Schur-complement invertibility hypotheses. The formalisation is complete in the repository state described here: the Lean source contains zero **sorry** or **admit**, and the headline results use only the standard axioms **propext**, **Classical.choice**, and **Quot.sound**.

1 Introduction

Trace logic for finite Markov chains studies what remains visible when a system has both observed and unobserved states. Given a transition matrix whose state space is decomposed as a disjoint sum of visible states and hidden states, one can eliminate the hidden block and obtain an effective transition operator on the visible block. For finite matrices this effective operator is the Schur-complement expression

$$P_{VV} + P_{VH}(1 - P_{HH})^{-1}P_{HV}.$$

In probabilistic language, this records direct visible-to-visible transitions together with excursions through hidden states before returning to the visible sector.

This note describes the Lean 4 formalisation in the public repository

github.com/velvetmonkey/trace-logic-lean.

The work is a companion note for a formal artefact, not a claim of new Markov-chain mathematics. The underlying identities are classical Schur-complement and censored-chain facts, and the motivation is close to Hoffman-Prakash trace logic: an observer sees a visible process while unobserved states are marginalised away. The contribution is the machine-checked, reusable formalisation in Mathlib style, together with an elementary proof route for transient hidden blocks that avoids Perron-Frobenius theory.

The repository is organised in three Lean modules. **TraceLogic/TraceLogic.lean** defines the basic trace map, row-stochasticity, nonnegativity, and entrywise trace order. **RequestProject/TraceLogic** proves the trace tower property and the transitivity of a trace-semantic order. **RequestProject/TraceLogic** proves that the required invertibility hypotheses follow for transient hidden blocks and derives unconditional transient corollaries.

2 Formal Setup

The base file works over finite matrices with real entries. The row-stochastic predicate is a bundled proposition.

```

structure IsRowStochastic {n : Type*} [Fintype n] (P : Matrix n n ℝ) :
  Prop where
  nonneg : ∀ i j, 0 ≤ P i j
  row_sum : ∀ i, ∑ j, P i j = 1

```

The nonnegativity predicate and trace map are:

```

def IsNonneg {m n : Type*} (M : Matrix m n ℝ) : Prop := ∀
  i j, 0 ≤ M i j

def traceMatrix {v h : Type*} [Fintype v] [Fintype h] [DecidableEq v] [
  DecidableEq h]
  (P : Matrix (v ⊕ h) (v ⊕ h) ℝ) : Matrix v v ℝ :=
  blockVV P + blockVH P * (1 - blockHH P)-1 * blockHV P

```

The block projections **blockVV**, **blockVH**, **blockHV**, and **blockHH** are submatrices induced by the sum injections. The formalisation also defines **matOpNorm**, the all-ones vector **ones**, and the entrywise order:

```

def trace_order {n : Type*} (A B : Matrix n n ℝ) : Prop := ∀
  i j, A i j ≤ B i j

```

Phase 5 adds two hidden-block hypotheses.

```

structure IsStrictlySubstochastic {n : Type*} [Fintype n]
  (A : Matrix n n ℝ) : Prop where
  nonneg : ∀ i j, 0 ≤ A i j
  row_sum_lt : ∀ i, ∑ j, A i j < 1

structure IsTransientHidden {n : Type*} [Fintype n] [DecidableEq n]
  (A : Matrix n n ℝ) : Prop where
  nonneg : ∀ i j, 0 ≤ A i j
  exists_N : ∃ N : ℕ, ∀ i, ∑ j, (A ^ N) i j < 1

```

Thus transience is formalised by a finite-power row-sum escape condition: some power of the hidden transition block is strictly substochastic.

3 Main Results

3.1 Trace Stochasticity

The basic Markov-chain theorem says that the Schur-complement trace of a row-stochastic matrix is row-stochastic when the hidden inverse exists and is nonnegative:

```

theorem trace_row_stochastic {v h : Type*} [Fintype v] [Fintype h]
  [DecidableEq v] [DecidableEq h]
  {P : Matrix (v ⊕ h) (v ⊕ h) ℝ} (hP : IsRowStochastic P)
  (hinv : IsUnit (1 - blockHH P))
  (hinv_nn : IsNonneg ((1 - blockHH P)-1)) :
  IsRowStochastic (traceMatrix P)

```

This is built from **trace_nonneg** and **trace_row_sum**. The row-sum proof uses the all-ones vector and the identity **inv_blockHV_mulVec_ones**.

3.2 The Trace Tower

The trace tower is the formal Schur-complement composition law. The file **RequestProject/TraceLogicPh** defines a reindexing operation and a two-step trace:

```

def reindexForTrace {v h1 h2 : Type*}
  (P : Matrix (v ⊕ (h1 ⊕ h2)) (v ⊕ (h1 ⊕ h2)) ℝ) :
  Matrix ((v ⊕ h2) ⊕ h1) ((v ⊕ h2) ⊕ h1) ℝ

def traceTwo {v h1 h2 : Type*} [Fintype v] [Fintype h1] [Fintype h2]
  [DecidableEq v] [DecidableEq h1] [DecidableEq h2]
  (P : Matrix (v ⊕ (h1 ⊕ h2)) (v ⊕ (h1 ⊕ h2)) ℝ) : Matrix v v ℝ

```

The headline theorem is:

```

theorem trace_tower {v h1 h2 : Type*} [Fintype v] [Fintype h1] [Fintype h2]
  [DecidableEq v] [DecidableEq h1] [DecidableEq h2]
  (P : Matrix (v ⊕ (h1 ⊕ h2)) (v ⊕ (h1 ⊕ h2)) ℝ)
  (hunit_h1 : IsUnit (1 - blockHH (reindexForTrace P)))
  (hunit_h2 : IsUnit (1 - blockHH (traceMatrix (reindexForTrace P))))
  (hunit_combined : IsUnit (1 - blockHH P)) :
  traceTwo P = traceMatrix P

```

It states that hiding h_1 and then h_2 gives the same visible matrix as hiding $h_1 \oplus h_2$ at once, provided the three Schur complements exist. The formal proof avoids importing a full 2×2 block-matrix inversion theorem. Instead it proves block-equation identities for the hidden equations, including `Y1_eq`, `Y2_schur_eq`, `Q_blockVV`, `Q_blockVH`, `Q_blockHV`, `Q_blockHH`, and `traceMatrix_eq_Q_plus_correction`.

3.3 Trace-Semantic Order

The trace-semantic order is an existential witness relation.

```

def trace_sem_le {n : Type*} [Fintype n] [DecidableEq n]
  (A B : Matrix n n ℝ) : Prop := ∃
  (k : ℕ) (P : Matrix (n ⊕ Fin k) (n ⊕ Fin k) ℝ),
  blockVV P = A ∧ traceMatrix P = B

```

Reflexivity uses the empty hidden block:

```

theorem trace_sem_le_refl {n : Type*} [Fintype n] [DecidableEq n]
  (A : Matrix n n ℝ) : trace_sem_le A A

```

The relation refines entrywise order under nonnegativity hypotheses:

```

theorem trace_sem_le_implies_trace_order {n : Type*} [Fintype n] [
  DecidableEq n]
  {A B : Matrix n n ℝ}
  {k : ℕ} {P : Matrix (n ⊕ Fin k) (n ⊕ Fin k) ℝ}
  (hVV : blockVV P = A) (htr : traceMatrix P = B)
  (hP_nn : IsNonneg P)
  (hinv_nn : IsNonneg ((1 - blockHH P)-1)) :
  trace_order A B

```

The transitivity theorem composes two hidden-state witnesses:

```

theorem trace_sem_le_trans {n : Type*} [Fintype n] [DecidableEq n]
  {A B C : Matrix n n ℝ}
  (hAB : trace_sem_le A B) (hBC : trace_sem_le B C)
  (hunit_AB : ∀(k : ℕ) (P : Matrix (n ⊕ Fin k) (n ⊕ Fin k) ℝ),
    blockVV P = A → traceMatrix P = B → IsUnit (1 - blockHH P))
  (hunit_BC : ∀(k : ℕ) (P : Matrix (n ⊕ Fin k) (n ⊕ Fin k) ℝ),
    blockVV P = B → traceMatrix P = C → IsUnit (1 - blockHH P)) :
  trace_sem_le A C

```

The proof constructs a combined witness on $n \oplus (\text{Fin } k_1 \oplus \text{Fin } k_2)$, then converts it to the required $\text{Fin}(k_1 + k_2)$ witness through `trace_sem_le_of_exists`.

3.4 Invertibility for Transient Hidden Blocks

The Phase 5 theorem `isUnit_one_sub_of_strictly_substochastic` proves the clean one-step case:

```
theorem isUnit_one_sub_of_strictly_substochastic {n : Type*} [Fintype n]
  [DecidableEq n]
  {A : Matrix n n ℝ} (hA : IsStrictlySubstochastic A) :
  IsUnit (1 - A)
```

The headline transient theorem is:

```
theorem isUnit_one_sub_of_transient {n : Type*} [Fintype n] [DecidableEq
  n]
  {A : Matrix n n ℝ} (hA : IsTransientHidden A) :
  IsUnit (1 - A)
```

This result discharges the invertibility side condition for the full transient class as formalised in the repository. The corresponding corollaries are:

```
theorem trace_tower_transient
  {v h1 h2 : Type*} [Fintype v] [Fintype h1] [Fintype h2]
  [DecidableEq v] [DecidableEq h1] [DecidableEq h2]
  (P : Matrix (v ⊕ (h1 ⊕ h2)) (v ⊕ (h1 ⊕ h2)) ℝ)
  (hHH_h1 : IsTransientHidden (blockHH (reindexForTrace P)))
  (hHH_Q : IsTransientHidden (blockHH (traceMatrix (reindexForTrace P))))
  )
  (hHH_combined : IsTransientHidden (blockHH P)) :
  traceTwo P = traceMatrix P
```

```
theorem trace_sem_le_trans_transient {n : Type*} [Fintype n] [DecidableEq
  n]
  {A B C : Matrix n n ℝ}
  (hAB : trace_sem_le A B) (hBC : trace_sem_le B C)
  (hunit_AB : ∀(k : ℕ) (P : Matrix (n ⊕ Fin k) (n ⊕ Fin k) ℝ),
    blockVV P = A → traceMatrix P = B →
    IsTransientHidden (blockHH P))
  (hunit_BC : ∀(k : ℕ) (P : Matrix (n ⊕ Fin k) (n ⊕ Fin k) ℝ),
    blockVV P = B → traceMatrix P = C →
    IsTransientHidden (blockHH P)) :
  trace_sem_le A C
```

These theorems are unconditional relative to the transient hidden-block hypotheses: the required `IsUnit` facts are proved inside Lean rather than assumed as theorem parameters.

4 A Perron-Frobenius-Free Invertibility Argument

The elementary transient argument is the most methodologically useful part of the formalisation. Classically one might prove invertibility of $1 - A$ from $\rho(A) < 1$, using Perron-Frobenius theory or spectral-radius estimates for nonnegative matrices. That route requires a library of spectral-radius facts for nonnegative matrices. The Lean development instead uses the finite-power row-sum condition in `IsTransientHidden`.

The strict case proves that if A is entrywise nonnegative and every row sum is less than 1, then $(1 - A).\text{mulVec}$ is injective:

```

theorem mulVec_one_sub_injective {n : Type*} [Fintype n] [DecidableEq n]
  {A : Matrix n n ℝ} (hA : IsStrictlySubstochastic A) :
  Function.Injective (1 - A).mulVec

```

The proof chooses an index where $|x_i|$ is maximal. If $x = Ax$, nonnegativity gives

$$|x_i| \leq \sum_j A_{ij} |x_j| \leq \left(\sum_j A_{ij} \right) |x_i| < |x_i|,$$

unless $x = 0$. Lean then converts injectivity of `mulVec` into `IsUnit (1 - A)` via Mathlib’s matrix column-independence machinery.

For the transient case, the source proves:

```

theorem IsNonneg_pow {n : Type*} [Fintype n] [DecidableEq n]
  {A : Matrix n n ℝ} (hA : ∀ i j, 0 ≤ A i j) (k : ℕ) : ∀
  i j, 0 ≤ (A ^ k) i j

theorem mulVec_one_sub_injective_of_transient {n : Type*} [Fintype n] [
  DecidableEq n]
  {A : Matrix n n ℝ} (hA : IsTransientHidden A) :
  Function.Injective (1 - A).mulVec

```

The proof obtains N such that A^N is strictly substochastic. It proves that $1 - A^N$ is invertible by applying the strict result to A^N . Then it uses the finite geometric factorisation

$$(1 - A)(1 + A + \cdots + A^{N-1}) = 1 - A^N$$

to conclude that $1 - A$ is invertible. This sidesteps Perron-Frobenius entirely. The spectral-radius formulation remains useful mathematical background, but it is not needed for the present Lean theorem.

5 Verification and Axioms

The Lean source files under `RequestProject/` and `TraceLogic/` contain zero occurrences of `sorry` or `admit`. The headline results were checked with `#print axioms`. Each of the following depends only on `propext`, `Classical.choice`, and `Quot.sound`: `trace_tower`, `trace_sem_le_trans`, `isUnit_one_sub_of_strictly_substochastic`, `isUnit_one_sub_of_transient`, `trace_tower_substochastic`, `trace_sem_le_trans_substochastic`, `trace_tower_transient`, `trace_sem_le_trans_transient`, `trace_row_stochastic`, `trace_sem_le_refl`, and `trace_sem_le_implies_trace_order`. No checked headline theorem depends on `sorryAx`.

6 Honesty About Scope

The mathematics formalised here is classical. Schur complements, censored finite Markov chains, and the quotient behaviour of eliminating hidden states are standard topics. This paper does not claim a new trace theorem for Markov chains. Its contribution is that the relevant finite-dimensional trace identities, witness composition, and transient-invertibility obligations have been made explicit, reusable, and machine certified in Lean 4.

The elementary transient closure is also not a new theorem in probability theory. Its value here is methodological: it avoids a heavier Perron-Frobenius dependency and fits the current Mathlib environment. This makes the formal development more robust and reduces the amount of upstream spectral theory needed before the trace-logic results can be used.

7 Motivating Connection to Witnesses

One motivation for trace logic is witness theory: an observer may see only a projection of a larger state space while hidden mechanisms continue to affect visible behaviour. The relation `trace_sem_le` makes this idea concrete by existentially quantifying a hidden-state witness matrix whose trace is the observed operator. The paper does not prove philosophical or epistemic claims about observers. It only formalises the matrix-level mechanism by which unobserved finite states can be marginalised and composed.

8 Conclusion

`trace-logic-lean` gives a complete zero-sorry Lean 4 formalisation of finite trace logic for Markov chains in the Schur-complement setting. The main certified results are row-stochasticity of the trace, the trace tower property, trace-semantic order transitivity, and transient-hidden-block corollaries that remove explicit invertibility hypotheses. The development uses standard Mathlib axioms only and avoids Perron-Frobenius theory for the transient invertibility theorem. Future work could connect the finite-power row-sum transience predicate to spectral-radius and Perron-Frobenius formulations as Mathlib’s nonnegative matrix theory grows.

References

- [1] M. Hoffman and P. Prakash. Trace logic for probabilistic systems. Cited here as background motivation for the trace-logic viewpoint.
- [2] C. D. Meyer. *Matrix Analysis and Applied Linear Algebra*. SIAM, 2000.
- [3] The Mathlib Community. The Lean mathematical library. <https://github.com/leanprover-community/mathlib4>.
- [4] Ben Cassie. `trace-logic-lean`. <https://github.com/velvetmonkey/trace-logic-lean>.